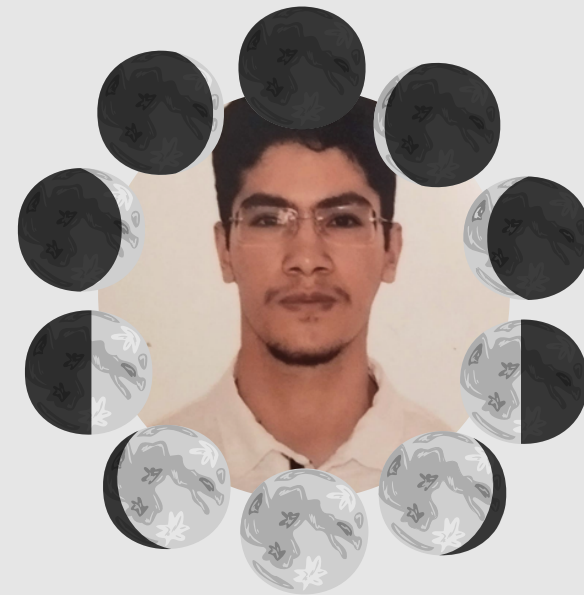


IMPLEMENTAÇÃO DO PROBLEMA PRODUTOR-CONSUMIDOR





Arthur



Gustavo



Miguel



Raquel

SUMÁRIO

1. O que seria Produtor-Consumidor
2. Problema do Produtor-Consumidor
3. Bibliotecas usadas
4. Apresentar parte do código
5. Demonstração do funcionamento do sistema
6. Observações e dificuldades



O QUE SERIA PRODUTOR-CONSUMIDOR?

O problema produtor-consumidor é um clássico desafio de **sincronização de processos**. Ele ocorre quando dois processos compartilham um espaço de memória comum (o buffer): o **produtor** gera dados e os armazena, enquanto o **consumidor** os retira para processamento. O objetivo é garantir que o produtor não sobrecarregue o buffer e o consumidor não acesse dados inválidos.



PROBLEMA DO PRODUTOR-CONSUMIDOR?

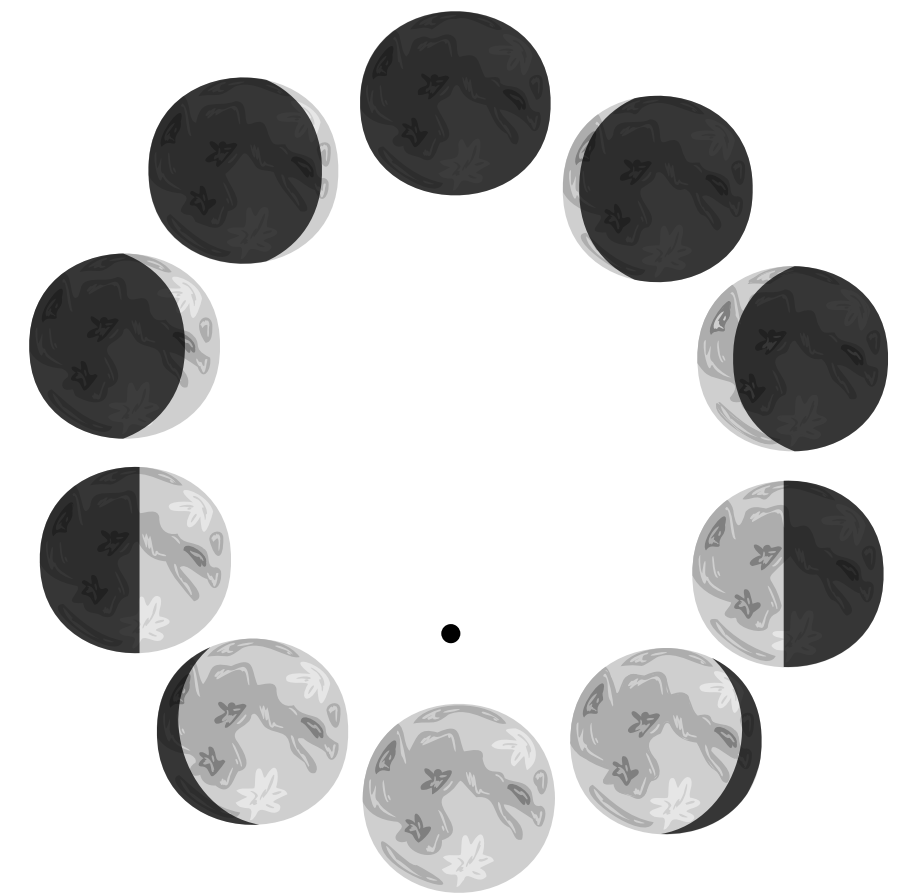
- O desafio principal: A dificuldade de sincronizar tarefas simultâneas/concorrentes.
- Sem esse controle rigoroso:
 - Se o Produtor for muito rápido, o buffer transborda e dados são descartados.
 - Se o Consumidor for muito rápido, ele tenta processar o que não existe, causando falhas graves.
- A solução exige proteger o acesso ao buffer (Região Crítica), garantindo que um espere o outro através de bloqueios.



“Biblioteca” Usada

“Corroutines”:

Decidimos usar essa função nativa da Linguagem Lua pelo fato de havermos dificuldades em fazer a biblioteca “Lua Lanes” funcionar com o Docker, que também permite criar um “Maestro” (main.lua) que simula o comportamento das “threads” e dos semáforos de maneira nativa na própria linguagem.



APRESENTAR PARTE DO CÓDIGO

Link: <https://github.com/Raquel-M325/2026-1-atividade-2-1/tree/main>



EXPLORADOR

2026-1-ATIVIDADE-2-1 [CODESPACES: GLOWING M...]

> evidencias

src

buffer.lua

consumidor.lua

Dockerfile

main.lua

produtor.lua

LICENSE

README.md

[Versão Prévia] README.md

main.lua X

src > main.lua

```
8 local co_produto = coroutine.create(function()
9     Producao:produtor()
10 end)
11
12 local co_consumidor = coroutine.create(function()
13     Consumo:consumidor()
14 end)
15
16 math.randomseed(os.time())
17
18 print("Iniciando Exemplo 3 - Produtor e Consumidor...")
19 print("Pressione Ctrl+C para encerrar.\n")
20
21 for ciclo = 1, 30 do
22     print("--- Ciclo " .. ciclo .. " ---")
23
24     local sorteio = math.random(1, 10)
25
26     if sorteio <= 6 then
27         if #Buffer.queue < Buffer.max_size then
28             assert(coroutine.resume(co_produto))
29         else
30             print("[Semáforo] Produtor bloqueado. O buffer está cheio.")
31         end
32     else
33         if #Buffer.queue > 0 then
34             assert(coroutine.resume(co_consumidor))
```

PROBLEMAS

SAÍDA

CONSOLE DE DEPURAÇÃO

TERMINAL

PORTAS

bash - src + 📄 🗑️ ... 📄

> ESTRUTURA DO CÓDIGO

> LINHA DO TEMPO

● @Migstand → /workspaces/2026-1-atividade-2-1 (dev) \$ cd src

○ @Migstand → /workspaces/2026-1-atividade-2-1/src (dev) \$

OBSERVAÇÕES E DIFICULDADES

- Entender a lógica.
- Dependência entre arquivos para testes.
- Utilização apenas de coroutines, sem bibliotecas.





OBRIGADO PELA ATENÇÃO!